Modular Infrastructure-as-Code for AWS Multi-Account Environments

A study of Terraform module composition, drift detection, and least-privilege IAM policy generation

Manikanta Reddy Mandadhi

Senior Data Scientist — Independent Research

2026

Contents

1. Abstract

Production AWS deployments rely on Terraform for declarative infrastructure management; the open question for any non-trivial estate is how to organize modules so that the codebase stays tractable as the estate scales. We design and evaluate a five-stratum module organization (foundation, networking, security, platform, workload) on a synthetic multi-account environment with 47 distinct workloads. Module composition reduces lines-of-code by 62% relative to a flat-file baseline, and reduces blast radius of a single bad apply (mean services impacted) from 28 to 4. Drift detection via terraform plan invocation in CI catches 94% of out-of-band changes within one hour. Least-privilege IAM policy generation from CloudTrail logs (custom Python tool) produces policies that pass the AWS Access Analyzer policy validation suite with zero high-severity findings on 41 of 47 workloads.

Keywords: infrastructure as code, Terraform, drift detection, IAM, AWS Well-Architected

2. Introduction

Organizations standardizing on AWS face a recurring scaling challenge: their Terraform codebase grows faster than their estate, module boundaries blur, and least-privilege IAM degrades into permissive default policies. The Well-Architected Framework prescribes principles but does not prescribe code-organization patterns. The research problem is to evaluate a five-stratum module composition pattern against a flat-file baseline along three axes: codebase tractability (lines of code, depth of module reuse), blast radius (services impacted by a single faulty apply), and IAM hygiene (percentage of resources with auto-generated least-privilege policies).

2.1 Research Problem

Organizations standardizing on AWS face a recurring scaling challenge: their Terraform codebase grows faster than their estate, module boundaries blur, and least-privilege IAM degrades into permissive default policies. The Well-Architected Framework prescribes principles but does not prescribe code-organization patterns. The research problem is to evaluate a five-stratum module composition pattern against a flat-file baseline along three axes: codebase tractability (lines of code, depth of module reuse), blast radius (services impacted by a single faulty apply), and IAM hygiene (percentage of resources with auto-generated least-privilege policies).

2.2 Research Questions and Hypotheses

Research question: Does a five-stratum module composition reduce lines-of-code relative to a flat-file baseline at constant functionality?

Hypothesis: We hypothesize a 50-65% reduction by extracting shared concerns (VPC topology, IAM roles,

observability) into reusable modules.

Research question: Does the stratum boundary materially reduce the blast radius of a faulty apply?

Hypothesis: We expect mean blast-radius (services impacted by a randomly-injected faulty apply) to fall by at least a factor of 5.

Research question: Does CI-integrated drift detection catch out-of-band changes within an SLA acceptable to security review boards?

Hypothesis: We expect 90%+ of injected drift to be caught within one hour via hourly terraform plan in CI.

Research question: Does CloudTrail-driven least-privilege IAM policy generation reduce AWS Access Analyzer high-severity findings?

Hypothesis: We expect zero high-severity findings on at least 80% of workloads after generation, vs ~30% with hand-written policies.

3. Literature Review

3.1 Theories Grounding the Problem

1. **Modularity in Software Architecture (Parnas, 1972)** — Modules should be designed around hideable decisions; in IaC, those decisions are configuration choices specific to one stratum (networking, security, workload). The five-stratum split is an explicit application of Parnas's information-hiding principle. (Parnas (1972))

2. **Well-Architected Framework (AWS, 2024)** — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability define the design pillars; Terraform module organization should support these without compromise. (AWS (2024))

3. **Least Privilege (Saltzer & Schroeder, 1975)** — Every program and every user should operate using the least privilege necessary; in cloud IAM, this principle is operationalized via per-resource scoped policies derived from observed access patterns. (Saltzer & Schroeder (1975))

4. **GitOps Operating Model (Beyer et al., 2016)** — Infrastructure changes should flow through the same review and merge processes as code changes; declarative IaC plus CI-driven applies is the canonical implementation. (Beyer et al. (2016))

5. **Drift Detection Theory** — Out-of-band changes diverge declared from actual state; the recovery cost grows with detection latency, motivating the SLA framing of the third research question. (industrial framing)

3.2 Supporting Examples

- AWS's published reference architectures (Landing Zone, Control Tower) embody parts of the five-stratum split; this work generalizes them into a self-contained pattern.
- Spacelift, Atlantis, and Env0 (commercial Terraform CI tooling) implement drift detection as a paid feature; this work demonstrates feasibility on stock GitHub Actions.
- Common Fate's AccessHandler illustrates programmatic least-privilege generation; the open-source version in this work is functionally equivalent for the most common access patterns.

4. Research Method

We construct two parallel implementations of the same 47-workload synthetic environment: a flat-file baseline (one .tf file per workload, no module reuse) and a five-stratum module-composed version. Both produce identical terraform plan output. We measure lines of code, module-reuse depth, time-to-apply, and inject 100 synthetic faulty applies to measure blast radius. Drift detection is evaluated by injecting 50 out-of-band changes via the AWS console and measuring CI-detection latency. Least-privilege IAM generation processes 30 days of synthetic CloudTrail logs and emits scoped policies; AWS Access Analyzer is run for validation.

5. Data Description

Source: Synthetic AWS multi-account environment plus 30 days of CloudTrail traces — Generated by simulator scripts in this repository

Coverage: 47 workloads × 2 implementations = 94 deployments; 12.4 million CloudTrail events; 100 fault-injection runs

Schema (selected fields):

- workload_id, stratum_split, implementation_variant
- tf_file_count, line_count, module_dependency_depth
- blast_radius_metric per fault_injection

Preprocessing: Synthetic CloudTrail events were generated to match the distribution of public cloud-config drift datasets; events were mapped to IAM action templates via the Service Authorization Reference.

License / availability: Synthetic.

6. Analysis

6.1 Codebase tractability

Lines of code, file count, and reuse depth across the 47-workload estate.

Variant	Total LOC	Files	Mean reuse depth
Flat-file baseline	39,140	411	1.0
Stratum-composed	14,820	112	3.7

6.2 Blast-radius simulation

100 synthetic faulty applies; blast-radius is the count of services impacted before terraform-plan validation rejects the change.

Variant	Mean services impacted	p95	Max
Flat-file baseline	28	63	112
Stratum-composed	4	11	19

6.3 Drift detection

50 injected out-of-band changes; detection latency measured via hourly terraform plan in CI.

Detection window	Coverage	False positive rate
<1 hour	0.94	0.02
1-2 hours	0.04	0.01
>2 hours	0.02	0.00

6.4 IAM Access Analyzer findings

Auto-generated policies vs hand-written baseline.

Variant	Workloads with 0 high findings	Mean findings/workload
Hand-written baseline	13/47 (28%)	3.8
Auto-generated	41/47 (87%)	0.4

7. Discussion

All four hypotheses are supported. The 62% LOC reduction is modestly above the 50-65% predicted band; the blast-radius reduction is exactly in line. Drift detection at 94% coverage within an hour is actionable for security review; the residual 6% are typically tag modifications that fall outside the resource scope of the plan. The IAM auto-generation result is the most surprising one in practical terms: it demonstrates that the typical 'permissive default' IAM posture is fully avoidable in routine deployments.

8. Conclusion

A five-stratum Terraform module composition delivers measurable improvements on codebase tractability, blast radius, drift detection, and IAM hygiene. The reference implementation, synthesized environment, and validation tooling are released as a reusable starting point for organizations standardizing on AWS.

9. Future Work

- Validate the pattern on real (not synthetic) multi-account estates with consenting teams.
- Add cost-anomaly detection across the same module strata.
- Explore Terraform-Pulumi hybrid for workloads whose configuration is cleaner in a general-purpose language.
- Extend the IAM generator to support data-perimeter policies (SCP) automatically.

10. References

1. Brikman, Y. (2022). *Terraform: Up & Running* (3rd ed.). O'Reilly.
2. AWS Well-Architected Framework. Amazon Web Services. https://aws.amazon.com/architecture/well-architected/
3. Parnas, D. L. (1972). *On the Criteria to be Used in Decomposing Systems into Modules.* CACM 15(12). https://dl.acm.org/doi/10.1145/361598.361623
4. Saltzer, J. H. & Schroeder, M. D. (1975). *The Protection of Information in Computer Systems.* Proceedings of the IEEE 63(9). https://www.cs.virginia.edu/~evans/cs551/saltzer/
5. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering.* O'Reilly. https://sre.google/sre-book/